

Jeu d'essai & restauration de la base

Base de données de test de **Petit Rayon de Soleil** : un jeu d'essai complet couvrant tous les cas de figure, et une procédure permettant de la **restaurer à l'identique** en une commande en cas d'incident.

Projet Petit Rayon de Soleil Auteur de Breyne Hélio Insertion db.js → seed()

Restauration npm run init-db

1
compte admin

2
comptes utilisateurs

5
messages validés

2
messages en attente

1
message rejeté

2
favoris

1 Comptes utilisateurs

Un administrateur (accès à la modération) et deux utilisateurs standards. Les mots de passe sont hachés en base (bcrypt) ; ils sont rappelés ici en clair uniquement à des fins de test.

ID	Rôle	Nom	Email	Mot de passe (test)
1	admin	Admin	admin@soleil.fr	admin123
2	user	Utilisateur Démo	user@soleil.fr	user1234
3	user	Camille Martin	camille@soleil.fr	camille1

2 Messages (les trois statuts)

Le champ `status` peut valoir `approved`, `pending` ou `rejected`. Le jeu d'essai contient les trois, pour couvrir tout le cycle de modération.

ID	Statut	Contenu	Proposé par
1	approved	Ne sois pas trop dur(e) envers toi-même...	seed
2	approved	Tu mérites tout l'amour et le respect...	seed
3	approved	Chaque petit pas compte...	seed
4	approved	Prends le temps de respirer...	seed
5	approved	Ta présence rend le monde un peu plus doux...	seed
6	pending	Tu es capable de bien plus que tu ne l'imagines.	user #2
7	pending	Ce petit rayon de soleil, c'est toi aujourd'hui.	user #3
8	rejected	Message hors sujet refusé par la modération.	user #3

Visibilité selon le statut. Seuls les messages `approved` sont renvoyés par l'API publique (`GET /api/messages`). Les `pending` apparaissent uniquement dans la file de modération de l'admin. Les `rejected` sont conservés en base pour la traçabilité, mais ne sont ni publiés ni reproposés à la modération.

3 Favoris

La table de liaison `favoris` relie un utilisateur à des messages validés (relation plusieurs-à-plusieurs). L'utilisateur de démo en a deux.

Utilisateur	Message favori	Contenu
#2 Utilisateur Démo	message #1	Ne sois pas trop dur(e) envers toi-même...
#2 Utilisateur Démo	message #3	Chaque petit pas compte...

4 Restauration / réinitialisation

En cas d'incident (base corrompue, données de test polluées par des essais manuels) ou pour faire repartir les tests de zéro, un script dédié remet l'application dans un état propre et connu — **en une seule commande, avec vérification automatique**.

Restaurer en une commande : `npm run db:reset`

Le script `scripts/reset-db.js` enchaîne les quatre étapes exigées et refuse de s'exécuter si une base de production Turso est configurée (garde-fou anti-effacement). Voici sa sortie réelle :

```
bash – npm run db:reset
```

```
$ npm run db:reset

[1/4] Suppression de la base locale... OK (1 fichier(s) retiré(s))
[2/4] Création des tables (init)... OK
[3/4] Réinjection des données de démonstration (seed)... OK
[4/4] Vérification :
      ✓ admin@soleil.fr (admin)
      ✓ user@soleil.fr (user)
      ✓ camille@soleil.fr (user)
      ✓ messages validés : 5 (attendu 5)
      ✓ messages en attente : 2 (attendu 2)
      ✓ messages rejetés : 1 (attendu 1)
      ✓ favoris          : 2 (attendu 2)

Base restaurée avec succès. Tous les comptes et volumes sont conformes.
```

Les quatre étapes du script

- 1 **Suppression de la base locale** — efface `soleil.db` et ses fichiers annexes (`-wal` , `-shm`).
- 2 **Relance de l'initialisation** — recrée les tables via `init()` .
- 3 **Réinjection des données de démonstration** — insère le jeu d'essai via `seed()` .
- 4 **Vérification des comptes de test** — contrôle que les 3 comptes attendus sont présents avec le bon rôle, et que les volumes (5 validés, 2 en attente, 1 rejeté, 2 favoris) sont exacts. Le script sort en erreur (`exit 1`) si un seul contrôle échoue.

NOTE Fermer d'abord le serveur (`npm start`) : sous Windows, un serveur en cours verrouille `soleil.db` et empêche sa suppression.

Méthode manuelle équivalente

Sans le script, les mêmes étapes se font à la main :

bash – restauration manuelle

```
# 1. Supprimer la base actuelle (et ses fichiers annexes WAL/SHM)
rm -f soleil.db soleil.db-wal soleil.db-shm

# 2. Recréer les tables + réinsérer le jeu d'essai complet
npm run init-db
```

Comment fonctionne la restauration

La commande `npm run init-db` exécute `node db.js --init`, qui appelle `init()` (création des tables via `CREATE TABLE IF NOT EXISTS`) puis `seed()` (insertion du jeu d'essai). La fonction `seed()` est **idempotente** : si des comptes existent déjà, elle ne réinsère rien — d'où l'étape 1 qui repart d'une base vide.

db.js – garde-fou d'idempotence

```
async function seed() {
  const count = await db.prepare('SELECT COUNT(*) AS n FROM users').get();
  if (count && count.n > 0) return; // déjà peuplée → on ne duplique pas
  // ... insertion des comptes, messages et favoris ...
}
```

Cas de la base en ligne (Turso)

La même commande initialise la base de production, en la préfixant des variables d'environnement Turso :

bash – initialisation de la base Turso

```
TURSO_DATABASE_URL=libsql://<base>.turso.io \
TURSO_AUTH_TOKEN=<jeton> \
npm run init-db
```

✓ **Résumé**

Élément demandé	Présent dans le jeu d'essai
Compte admin	oui admin@soleil.fr
Compte utilisateur	oui 2 comptes (user@soleil.fr , camille@soleil.fr)
Messages validés	oui 5 messages approved
Messages en attente	oui 2 messages pending
Messages rejetés	oui 1 message rejected (conservé)
Favoris	oui 2 favoris
Procédure de réinitialisation	oui script npm run db:reset (suppression + init + seed + vérification)
Vérification des comptes de test	oui automatisée par le script (échoue en exit 1 si non conforme)

Petit Rayon de Soleil — Annexe « Jeu d'essai & restauration ». Données et sortie du script vérifiées sur l'exécution réelle de `npm run db:reset`.